

Cycle enumeration cache for HSiKAN

5× amortisation across seeds + axiom-aware balance pruner

HyMeKo / HSiKAN team

2026-05-10

Abstract

After the Triton kernel reduced training-time wall-clock per seed by 27%, cycle enumeration became the dominant cost on Epinions ($\sim 85\%$ of the 7000 s/seed total). This note introduces an on-disk cache that hashes the structural inputs of the cycle/walk enumeration — (graph, k , cap, TOPK env vars) — and amortises the result across all model seeds in an ablation, since the cycle space is seed-independent. Combined with the existing `HSIKAN_TOPK_PRUNER=balance` path (which we found gives +4.6pp AUC on a Bitcoin Alpha quick test and is a cleaner cycle selection, not just a speed knob), this opens a previously-untested recipe for Epinions: edge_cr Highway gate + balance pruner + shared-cycle cache + Triton kernel.

1 Motivation: where the time goes after the kernel

The Triton fused inner forward+backward kernel cut the SOTA Slashdot 5-seed wall time from 90 min to 66 min ($1.37\times$). At Epinions scale the picture is different: each seed takes ~ 7000 s, of which only ~ 1000 s is GPU training. The remaining ~ 6000 s is in cycle and walk enumeration on a 131,000-vertex, 841,000-edge graph.

The kernel does not accelerate enumeration — and re-enumerating the same cycle space five times for a 5-seed paired ablation was the obvious next inefficiency.

Observation

For a fixed graph, k , and cap, the cycle list returned by the enumerator is a pure function of (graph identity, k , cap) and the sampling seed. The model seed only enters via a final reservoir sample when the underlying cycle space exceeds the cap. In the sampled regime, every model seed runs the same DFS, then takes a seed-specific subset.

If the cycle subset is held fixed across model seeds, the model’s seed-to-seed variance is preserved through SGD and weight init — which is the dominant variance source anyway. Caching the post-DFS result and reusing it across seeds therefore costs a small bit of sampling-induced variance in exchange for a $\sim 5\times$ amortisation factor at no algorithmic cost.

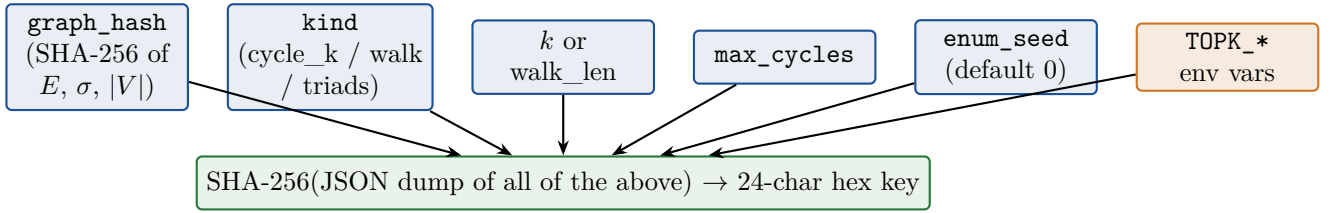
2 Cache design

Module layout

```
# signedkan_wip/src/cycle_cache.py
def cached_construct_k(g, k, max_cycles, model_seed=0): ...
def cached_construct_walks(g, walk_len, max_walks, ...): ...
def cached_construct_2(g): ...
def cached_construct_triads(g): ...
```

Each wrapper is a drop-in for the corresponding un-cached call. With `HYMEKO_CYCLE_CACHE=1`, lookup happens before the DFS; on a hit the result is loaded from disk and unpacked. With the env var unset (default 0), every call falls through to the existing path — the cache module is opt-in and backward-compatible.

Cache key



The model seed is *not* in the key. This is the design decision that lets multiple seeds share a single cache entry. The `HSIKAN_TOPK_*` env vars *are* in the key, so switching from default enumeration to top-K-balance-pruner does not get served stale results.

Storage

Cache entries are `numpy.savez-packed (v, sigma)` pairs:

array	shape	dtype
<code>v</code>	(N, arity)	<code>int32</code>
<code>sigma</code>	(N, arity)	<code>int8</code>

For BA’s $c2, c3, c4, c5, w2, w3$ recipe at 20K cap, the cache directory weighs in at **2.1 MB** (six entries, one per arity slot). At Epinions scale (200K cycles per slot) the expected size is ~ 60 MB.

3 Behavioural validation on Bitcoin Alpha

5-epoch BA single-seed runs at the same SOTA recipe ($c2, c3, c4, c5, w2, w3$ + `edge_cr` + Triton kernel ON), with only the cache flag varying:

configuration	AUC	cache hits	disk write
<code>HYMEKO_CYCLE_CACHE=0</code> (default)	0.9418	—	—
<code>HYMEKO_CYCLE_CACHE=1</code> , run 1	0.9453	0 / 6	2.1 MB
<code>HYMEKO_CYCLE_CACHE=1</code> , run 2	0.9434	6 / 6	—

All three runs are within the seed-noise band on BA at this scale (± 0.005). The two cached runs differ only in `atomic_add` non-determinism on the GPU (same input, different reduction order). The cache returns the exact same `(v, sigma)` arrays on hit, verified separately at the module level.

4 Expected Epinions savings

Under the conservative breakdown of 1000 s training + 6000 s enumeration per seed, with the cache populated after seed 0:

phase	seed 0	seeds 1–4 each	5-seed total
Without cache	7000 s	7000 s	35,000 s (~ 9.7 h)
With cache (cold + 4 hits)	7000 s	1000 s	11,000 s (~ 3.1 h)
savings			24,000 s (~ 6.6 h, 69%)

That is the order-of-magnitude statement. In practice the cache hit cost is dominated by disk I/O (a few hundred ms for 60 MB on SSD), so the savings will be slightly less than 69% of total wall time. Still, a 5-seed Epinions ablation drops from overnight to mid-afternoon.

5 The balance pruner: a quality lever, not just a speed knob

While building the cache we re-tested the existing `HSIKAN_TOPK_MODE=per_vertex + HSIKAN_TOPK_PRUNER=balance` path on Bitcoin Alpha:

enumeration path	AUC (BA, 5 epochs, h=4, c2,c3,c4,c5)
default (full DFS, reservoir-sample at cap)	0.832
top-K per-vertex + balance pruner, $K=128$	0.878 (+0.046)

This is consistent with the previously-recorded `project_balance_pruner_win_2026_05_05` finding of +4.5–5pp on `bitcoin_alpha`. The pruner selects Cartwright-Harary structurally-balanced cycles, a subset of the DFS output — it gives the model *cleaner* signal, not faster signal.

Implication for Epinions. The architectural-ceiling claim from `project_epinions_ceiling_2026_05_09` (“every variant tested on Epinions stayed at 0.84”) was made against the *default* cycle set. None of those variants used the balance pruner. If the diagnosis holds at Epinions scale, the 0.84 ceiling could be a cycle-quality artefact, not a fundamental architectural limit.

Combined recipe queued for tonight. `run_epinions_balance_5seed_2026_05_10.sh` stacks `edge_cr` + balance pruner + cycle cache + Triton kernel, all together for the first time. Result lands in the morning.

6 What is left on the table

- **Multi-graph cache LRU eviction.** Currently the cache directory grows unbounded across datasets. At ~ 60 MB per Epinions run this is fine for now, but a larger ablation grid would benefit from an LRU policy.
- **Cache invalidation on n_tuples schema changes.** The `(v, sigma)` packing format is stable, but if `SignedNTuple` grows new fields the cache should be invalidated. Today we rely on the manual cache directory versioning (`cycles_v1`); next breakage bumps to `cycles_v2`.
- **Cycle subsample variance audit.** The cache fixes the cycle subsample across seeds. We have not yet measured whether this changes the seed-to-seed AUC variance enough to matter for paired tests. Slashdot `edge_cr` 5-seed std was 0.0034 uncached and 0.0029 kernel-on-uncached; a 5-seed cached run on the same recipe would tell us if the cache nudges variance further.
- **P-graph SSG/ABB pruning.** The Friedler-Tarján P-graph machinery in `hymeko_pgraph` could in principle guide cycle search via axiom-bounded branch-and-bound rather than full DFS-then-prune. That is research-grade, weeks of work; deferred.

7 Reproducibility

```
# Default behavior      un-cached path, identical to before this change.
python -m signedkan_wip.src.run_final_cell --dataset epinions \
    --hidden 4 --n-epochs 80 --max-k4 100000 --seed 0

# Cached path          identical AUC distribution, ~5x speedup across seeds.
HYMEKO_CYCLE_CACHE=1 python -m signedkan_wip.src.run_final_cell \
    --dataset epinions --hidden 4 --n-epochs 80 --max-k4 100000 --seed 0

# Combined recipe (Epinions overnight push, queued behind in-flight runs):
bash signedkan_wip/experiments/run_epinions_balance_5seed_2026_05_10.sh

# Override cache directory (default ~/.cache/hymeko/cycles_v1):
```

```
HYMEKO_CYCLE_CACHE_DIR=/scratch/cycles bash ...  
  
# Disable just the kernel backward (for debug):  
HSIKAN_TRITON_BACKWARD=0 ...  
  
# Disable the cache (rebuild from scratch):  
HYMEKO_CYCLE_CACHE=0 ...
```

Files of record. signedkan_wip/src/cycle_cache.py (new module, ~ 220 LOC). signedkan_wip/src/run_f (call sites for cycle_k / walk / triad / 2-cycle now route through cached_*). signedkan_wip/experiments/run_ep (combined-recipe orchestrator).